

World Models

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

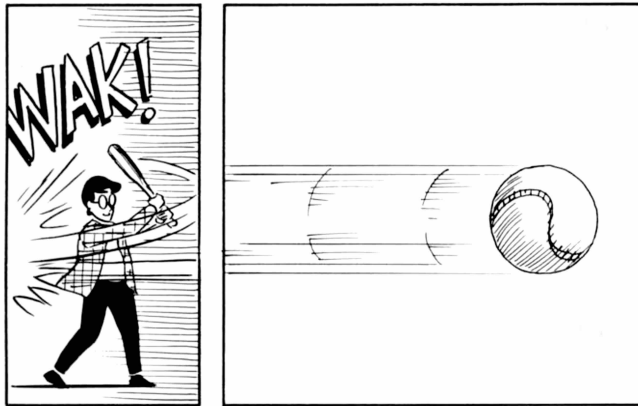
KAUST Academy
King Abdullah University of Science and Technology

July 6, 2026

1. Recap and background: world models so far, RL vocabulary, VAEs
2. World models: motivation, definition, applications, and modelling paradigms
3. Ha & Schmidhuber (2018): V-M-C architecture and training inside the dream
4. RSSM / PlaNet: stochastic and deterministic latents, ELBO, planning with CEM
5. Dreamer → DreamerV2 → DreamerV3: behaviors learned in imagination
6. Generative simulators (Sora, GAIA, Genie) vs. JEPA-style latent prediction
7. LeWorldModel (LeWM): end-to-end JEPA from pixels, SIGReg, and latent planning
8. Limitations, summary, hands-on resources, and references

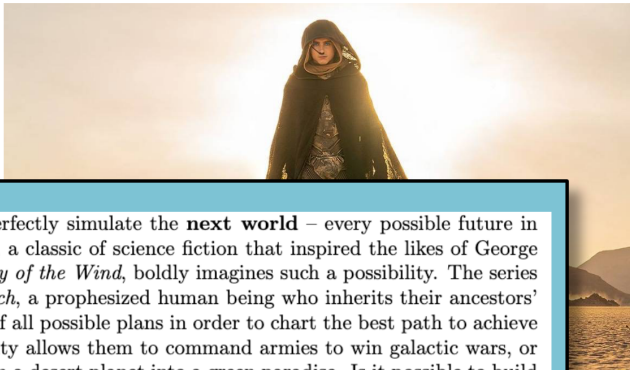
- ▶ **JEPA lecture:** a world model as a *predictor in representation space*; multistep and hierarchical world models; V-JEPA 2-AC planning with CEM over imagined latent futures.
- ▶ **Foundation-models lecture:** the large pretrained backbones (DINOv2, V-JEPA 2) that such systems build on.
- ▶ **Today, the control side of the story:** agents that learn a model of their environment, imagine futures inside it, and use those imagined futures to plan or to train a policy. This is **model-based reinforcement learning**.
- ▶ Route: Ha & Schmidhuber (2018) → PlaNet / RSSM → Dreamer v1–v3 → generative simulators vs. JEPA → LeWorldModel.

Intuition: a baseball batter reacts in milliseconds; the swing relies on a **predictive**, action-conditioned internal model, not on slow deliberate search.



Scott McCloud excerpt (time-space in comics).

Dune offers one notion of a world model...

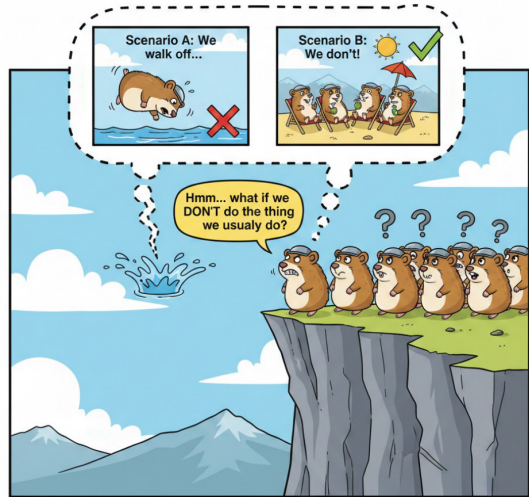


What would you do if you could perfectly simulate the **next world** – every possible future in the environment that we reside? *Dune*, a classic of science fiction that inspired the likes of George Lucas' *Star Wars* and Miyazaki's *Valley of the Wind*, boldly imagines such a possibility. The series is centered around the *Kwisatz Haderach*, a prophesized human being who inherits their ancestors' memories and simulates the outcomes of all possible plans in order to chart the best path to achieve their goals [25]. Such superhuman ability allows them to command armies to win galactic wars, or to oversee global-scale projects that turn a desert planet into a green paradise. Is it possible to build towards computer systems with similar functionalities using a similar approach?

Text from <http://arxiv.org/abs/2507.05169>

the psychology literature has long defined our notion of a world model as “hypothetical thinking”

informally, we speak about this as a “thought experiment”



- ▶ Humans develop a mental model of the world based on what they are able to perceive with their limited senses. Our decisions and actions are based on this internal model.
- ▶ Train a generative world model in an unsupervised way to learn a compact spatio-temporal representation of the environment.
- ▶ Use features from the world model to train a simple policy (agent) to solve the required task.

Definition: A *world model* takes a previous world state s and an action a , and samples (or predicts) the next world state s' through a distribution (or function):

$$s' \sim p(s' | s, a)$$

Question: What does a world state hold?

Answer: It depends on what you're trying to model. Consider the contrast of:

- A) All geopolitical entities, their leaders, and decisions they make
- B) Physical interactions in a 3D world

- ▶ A **world model** is a learned map that summarizes experience so we can predict (or sample) **what happens next** and sometimes **counterfactuals**: “if I had pushed left ...”
- ▶ An **agent** chooses actions a_t and receives observations o_t and rewards r_t . Planning and learning are easier if we can **imagine** plausible futures without acting in the real environment.
- ▶ Core tension: predict **everything in pixels** (high-dimensional, noisy) vs. predict in a **compact latent space** (lose detail but gain stability and generalization).

How could we use a World Model (WM)?

1. Generate (simulated) training data for robotics
2. Generate (simulated) training data for autonomous driving
3. Create interactive worlds (computer games) through prompting
4. Enable automatic generation / editing / refinement of 3D animation or graphics
5. Allow a thinking model to hypothesize about physical interactions in the real world
6. Enable faster creation of environments for virtual reality or augmented reality

Generate a 3D Scene, then Render/Simulate



Interactive Videos



Latent World Representations



CMU 10-423, Lecture 26.



IMAGE CREDITS: WORLD LABS

Fei-Fei Li's World Labs speeds up the world model race with Marble, its first commercial product

Rebecca Bellan — 5:44 AM PST · November 12, 2025

[World Labs](#), the startup founded by AI pioneer [Fei-Fei Li](#), is launching its first commercial world model product. [Marble](#) is now available via freemium and paid tiers that let users turn text prompts, photos, videos, 3D layouts, or panoramas into editable, downloadable 3D environments.

StrictlyVC an endeavor

May 27 | Athens, Greece

World Labs.

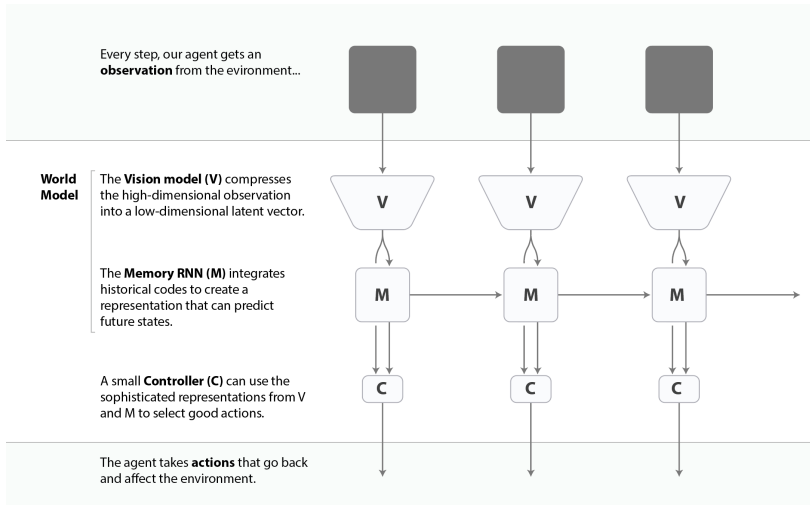
This is a computer vision course, so here is the RL vocabulary used today:

- ▶ **Agent–environment loop:** at each step the agent receives an observation o_t , picks an action a_t , and gets a reward r_t from the environment.
- ▶ **Return:** cumulative (discounted) reward over time; the quantity the agent maximizes.
- ▶ **Policy** $\pi(a | s)$: the agent's action rule. **Value function** $V(s)$: expected return from state s ; a network estimating it is called a **critic**, and the policy network an **actor**.
- ▶ **Model-free RL:** learn the policy and value directly from real interactions; no model of the environment.
- ▶ **Model-based RL:** first learn a model of the environment (the **world model**), then plan or train the policy inside it. Typically needs far fewer real interactions.

- ▶ An **autoencoder** compresses an input x to a latent code z (encoder) and reconstructs x from z (decoder).
- ▶ A **VAE** (Kingma & Welling, 2014) makes the latent *probabilistic*: the encoder outputs a distribution $q(z | x) = \mathcal{N}(\mu, \sigma^2)$ and z is **sampled** from it before decoding.
- ▶ Training maximizes the **ELBO** (evidence lower bound):

$$\mathcal{L} = \underbrace{\mathbb{E}_{q(z|x)}[\log p(x | z)]}_{\text{reconstruction}} - \underbrace{\text{KL}(q(z | x) \parallel \mathcal{N}(0, I))}_{\text{keep latents near a prior}}$$

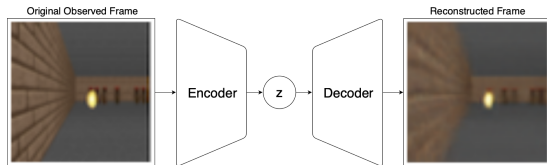
- ▶ The sampling step stays differentiable via the **reparameterization trick**: $z = \mu + \sigma \cdot \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, I)$.
- ▶ Why it matters today: Ha & Schmidhuber's vision module *is* this VAE, and RSSM training uses the same ELBO idea unrolled through time.



World model overview.

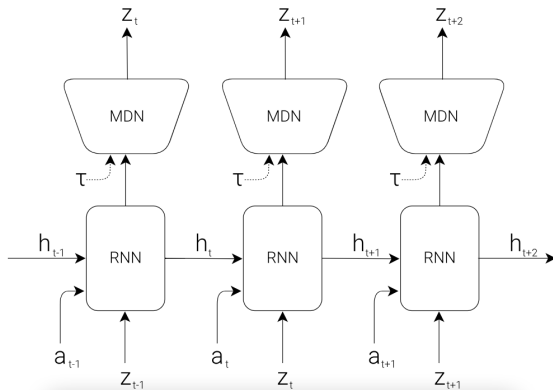
Source: Ha & Schmidhuber, 2018

- ▶ **Goal:** learn an abstract, compressed representation of each observed input frame.
- ▶ **VAE:** first stage of a two-stage sequence model; similar in spirit to modern tokenizers, but trained with the classic VAE objective (see the background slide).



Source: Ha & Schmidhuber, 2018

- ▶ Model $p(z_{t+1} | z_t, a_t, h_t)$ where z_t is the latent state at time t , a_t is the action at time t , and h_t is the hidden state at time t .
- ▶ **MDN-RNN**: an RNN (LSTM, from the RNN lecture) whose output parameterizes a **mixture of Gaussians** over the next latent z_{t+1} ; a mixture can represent several plausible futures at once.
- ▶ A temperature parameter τ scales the sampling uncertainty; it becomes important when training inside the dream.



MDN-RNN architecture.

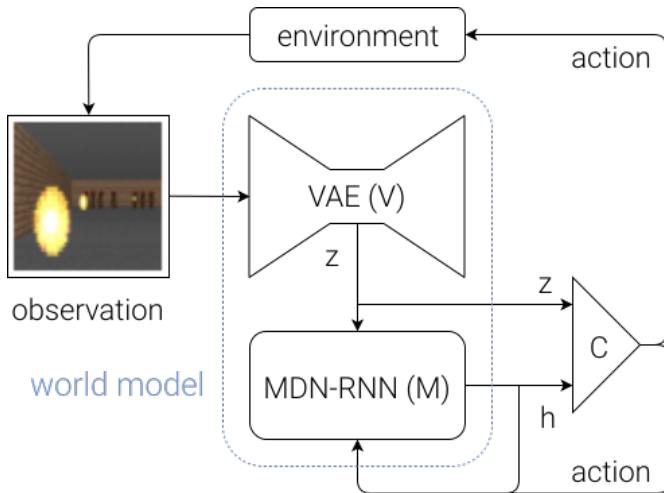
Source: Ha & Schmidhuber, 2018

- ▶ The **Controller (C)** selects actions to maximize expected cumulative reward during agent rollout.
- ▶ Designed to be as simple and small as possible; trained **separately** from the Vision (V) and Memory (M) modules.
- ▶ Most agent complexity is thus concentrated in the world model (V and M).
- ▶ C is a **single-layer linear** model that maps the latent vector z_t and RNN hidden state h_t directly to the action a_t at each time step:

$$a_t = W_c[z_t; h_t] + b_c$$

- ▶ W_c and b_c are the weight matrix and bias; $[z_t; h_t]$ is the concatenated latent and memory state, and a_t the resulting action.
- ▶ Because C has only a few hundred parameters, it is trained with **CMA-ES**, an evolution strategy that searches weight space directly; no gradients through the agent loop are needed.

World model: Putting Everything Together



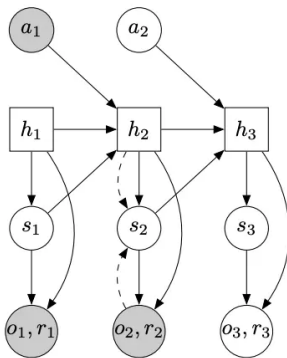
Agent-environment flow (observation $\rightarrow z_t$, RNN state h_t , action a_t , next step).

Source: Ha & Schmidhuber, 2018

- ▶ The memory model M can generate its own environment: replace real observations with imagined latents and train the controller **entirely inside the hallucinated rollout**.
- ▶ In the VizDoom experiment, a policy trained purely in the dream **transfers back to the real game** (as reported in the paper).
- ▶ The failure mode: the controller learns to **exploit model errors**, steering into states the model predicts wrongly (for example, imagined worlds where enemies never fire).
- ▶ Countermeasure: raise the temperature τ so the dream is noisier and harder to exploit; the resulting policy is more robust in the real environment.
- ▶ This exploit-the-model problem returns in Dreamer, handled there with KL constraints on the latent dynamics.

- ▶ **Separation of concerns:** perception (VAE), dynamics (RNN), decision (controller). Easy to prototype, but the modules do **not** share gradients end-to-end through the full agent loop at scale.
- ▶ **Stochastic latents** matter: the world is multi-modal; Gaussian or MDN heads encode “many possible futures”.
- ▶ Next step (PlaNet, Dreamer): unify training with **RSSM**-style latent dynamics and **model-based RL** losses.

- ▶ Purely deterministic RNNs (e.g., GRUs, LSTMs) struggle to represent multiple plausible futures in complex, stochastic environments.
- ▶ **Recurrent State Space Models (RSSM)** address this by combining:
 - **Deterministic hidden state** h_t (e.g., GRU): maintains long-term memory and context.
 - **Stochastic state** z_t (e.g., Gaussian): captures uncertainty and enables diverse possible futures.
- ▶ This split allows RSSM to unify stable memory (for causality) and flexible modeling (for multi-modal, uncertain environments).
- ▶ Without the stochastic component, agents often **fail to learn** in partially observable or noisy tasks.
- ▶ Result: a flexible temporal representation for RL agents capable of handling uncertainty and representing many possible outcomes.



Recurrent State Space Model (RSSM). Circles are stochastic variables (filled when observed) and squares are deterministic variables; solid lines denote the generative process, dashed lines the inference model. A mixed deterministic–stochastic model.

Source: [PlaNet](#), [arXiv:1811.04551](#).

- **Posterior transition (training):** Updates stochastic state using real observation

$$z_t \sim q_\phi(z_t \mid h_t, x_t)$$

- **Prior transition (imagination):** Predicts next latent state without observation,

$$z_t \sim p_\theta(z_t \mid h_t)$$

- **Training objective:** the model learns by reconstructing images, predicting rewards, and keeping its imagined predictions close to what really happened, using an ELBO-style loss (next slide).
- This ensures that planning in latent space is meaningful: predictions made during “imagination” are grounded in real transitions.

The same ELBO as the VAE, unrolled through time, with a reward head added:

$$\max \mathbb{E}_q \left[\sum_t \left(\underbrace{\ln p(x_t | h_t, z_t)}_{\text{reconstruction}} + \underbrace{\ln p(r_t | h_t, z_t)}_{\text{reward prediction}} - \beta \underbrace{\text{KL}(q(z_t | h_t, x_t) \| p(z_t | h_t))}_{\text{imagination matches reality}} \right) \right]$$

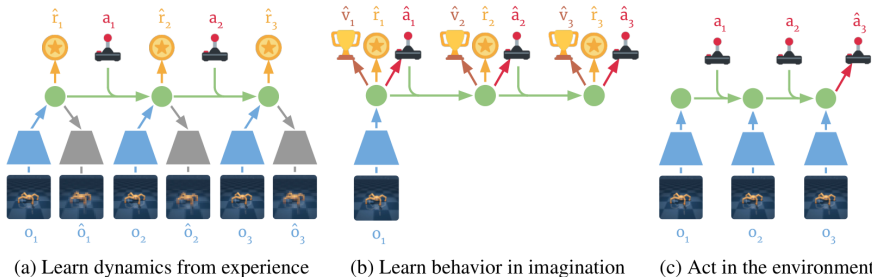
- ▶ **Reconstruction** grounds the latent state in observations.
- ▶ The **reward head** makes latent rollouts useful for planning and RL.
- ▶ The **KL term** pulls the prior (used during imagination) toward the posterior (informed by real observations), so imagined futures stay realistic.

- ▶ **PlaNet** (Hafner et al., 2019) pairs the RSSM with **online planning**; there is no policy network at all.
- ▶ At each step: sample candidate action sequences, roll them out in **latent space**, score them with the predicted rewards, and refine with the **cross-entropy method (CEM)**; execute the first action and replan.
- ▶ This is the same CEM / model-predictive-control loop V-JEPA 2-AC used in the JEPA lecture; PlaNet did it with an ELBO-trained generative model and a reward head instead of goal images.
- ▶ Learns image-based control tasks with far fewer environment interactions than model-free baselines.
- ▶ Two costs: planning at every step is expensive, and CEM has no value estimate beyond its horizon. **Dreamer** fixes both.

Model-based RL agent with an RSSM core that reaches state-of-the-art performance on image-based control tasks while requiring fewer samples.



Hafner et al., *Dream to Control*. Chen, *Towards AI* (2020).

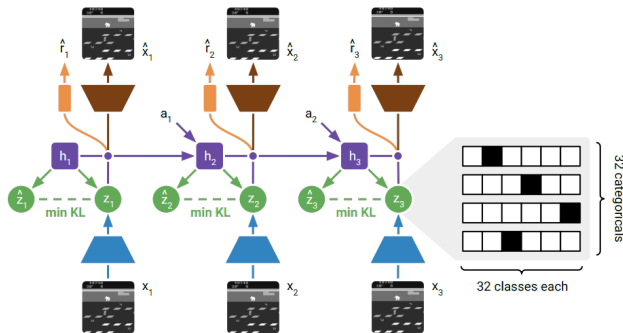


Dreamer architecture. Diagram credit: Hafner et al. (ICLR 2020), [PDF](#)

Dreamer is composed of three main parts:

- **(a) Dynamics Learning**: learns representation, transition, reconstruction, reward models.
- **(b) Behavior Learning**: based on the dynamics model, learns actor-critic architecture to maximize rewards.
- **(c) Environment Interaction**: using action model for data collection.

- ▶ Instead of planning at act time (PlaNet), Dreamer trains an **actor** and a **critic** on **imagined latent trajectories**.
- ▶ Rollouts never touch pixels: start from latent states of replayed real experience, then unroll the RSSM **prior** for a short horizon.
- ▶ The **critic** estimates value beyond the horizon; the **actor** is improved by backpropagating value gradients **through the learned dynamics**, which is possible because the world model is differentiable.
- ▶ Result: acting is a single forward pass (no per-step search), and the critic gives long-horizon credit assignment that CEM planning lacked.



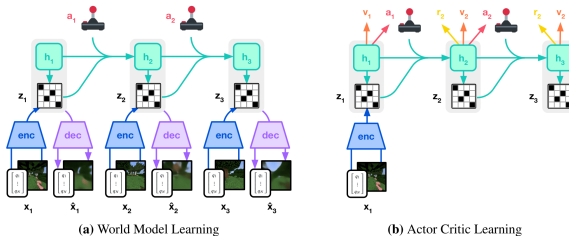
World-model overview. Diagram credit: Belotti (2023)

- **Key upgrade:** The world model now uses **discrete categorical latents** instead of continuous Gaussian variables. Each latent consists of 32 variables, each taking one of 32 categories, enabling a richer and more expressive latent space.
- Discrete latents are optimized using **straight-through gradients** for effective backpropagation despite non-differentiability. This allows for more precise representation and imagination.
- **KL balancing** is introduced to stabilize training by balancing the KL divergence term across model components.

- ▶ Achieves state-of-the-art performance across 150+ diverse tasks with a single configuration.
- ▶ Learns a world model to imagine future trajectories. Successfully solves challenging open-world environments such as collecting diamonds in Minecraft without human data.
- ▶ Architecturally similar to DreamerV2. Robustness and scalability stem from careful normalization, balancing, and mathematical tricks that enable stability across domains.

Learning algorithm consists of three neural networks:

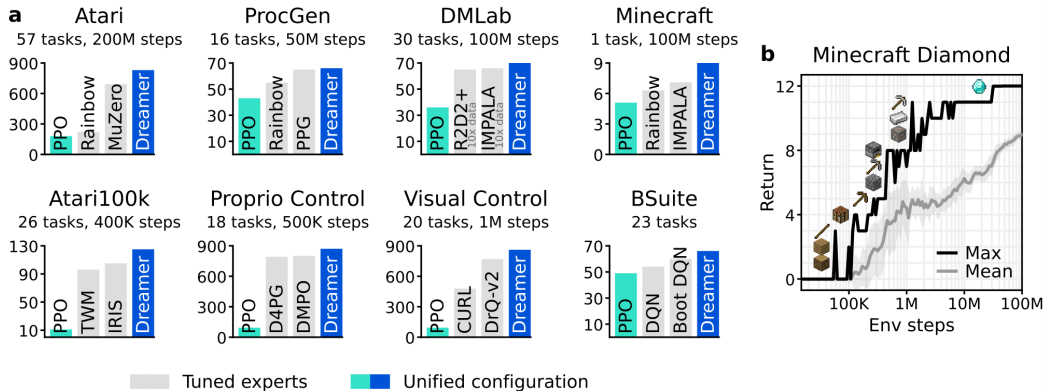
- **World model:** RSSM model that predicts the outcomes of potential actions, modeling future states of the environment.
- **Critic:** Judges the value of each predicted outcome, estimating how beneficial a state is for achieving the agent's goals.
- **Actor:** Selects actions intended to achieve the most valuable outcomes by maximizing the evaluated value.



Hafner et al., [arXiv:2301.04104](https://arxiv.org/abs/2301.04104).

The “mathematical tricks” behind one configuration for 150+ tasks:

- ▶ **Symlog predictions:** inputs and regression targets pass through a signed-log transform, taming value scales that differ by orders of magnitude across domains.
- ▶ **Two-hot return targets:** the critic predicts a distribution over discretized returns instead of a single scalar, which is more robust to outliers.
- ▶ **KL balancing and free bits:** the world-model KL term is split and clipped so neither prior nor posterior collapses, and small KL values are not over-optimized.
- ▶ **Percentile return normalization:** rewards are normalized by a running percentile range, so exploration behaves consistently across dense and sparse rewards.
- ▶ Published in *Nature* (2025); the headline demo is collecting diamonds in Minecraft from scratch, without human data.



Benchmark summary: with fixed hyperparameters across all domains, Dreamer outperforms tuned expert algorithms across a wide range of benchmarks and data budgets, and learns to obtain diamonds in Minecraft from scratch given sparse rewards.

Hafner et al., [arXiv:2301.04104](https://arxiv.org/abs/2301.04104).

- ▶ Large video generators are increasingly pitched as world simulators; most build on diffusion or autoregressive generation (covered in the diffusion lecture of the full course).
- ▶ **Sora** (OpenAI, 2024) frames video generation explicitly as “world simulation”; **GAIA-1** and its successors (Wayve) generate driving scenarios for autonomous-vehicle development.
- ▶ **Genie** (Google DeepMind) makes **playable** worlds: the model predicts the next frames *given user actions*, which is exactly the world-model interface, realized in pixel space. Genie 3 (2025) is reported to run interactive worlds in real time at 720p.
- ▶ The debate: photorealistic video is not the same as physically consistent prediction. Failures in object permanence and physics are common, and evaluating a simulator’s “correctness” is an open problem.

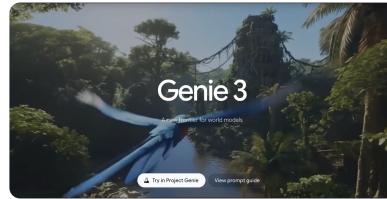
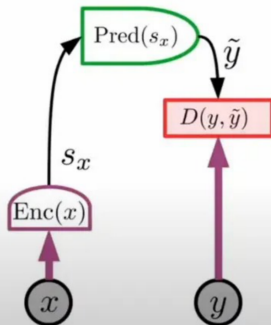
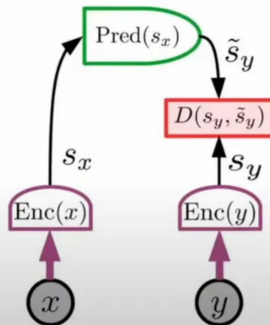


Image credit: CS25.

- ▶ **Generative:** predicts y (with all the details, including irrelevant ones)
- ▶ **Joint Embedding:** predicts an **abstract representation** of y

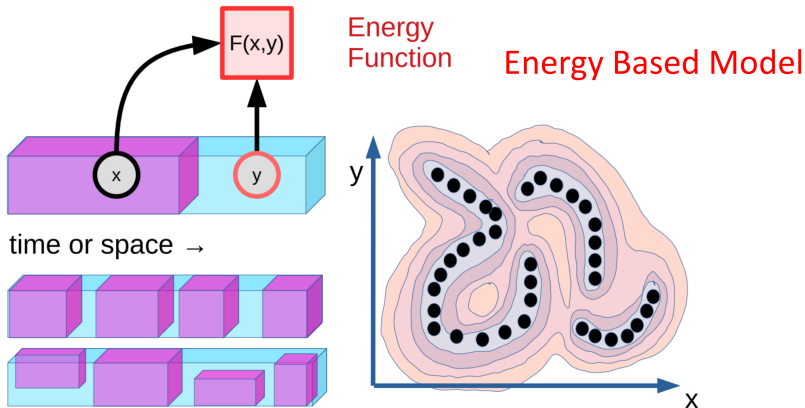


a) Generative Architecture
Examples: VAE, MAE...

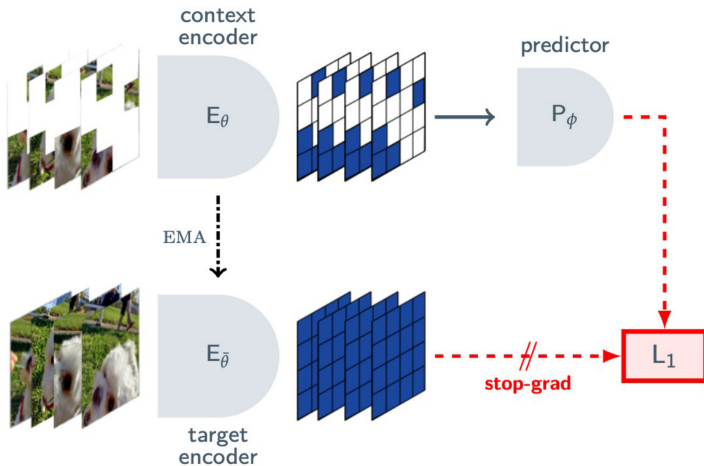


b) Joint Embedding Architecture

Image credit: CS25.

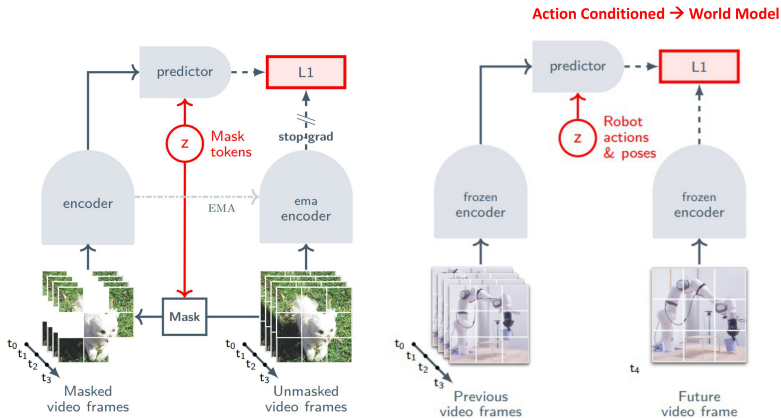


Paper: [OpenReview](#), Y. LeCun (2022).



Paper: Bardes et al., [arXiv:2404.08471](https://arxiv.org/abs/2404.08471) (V-JEPA).

V-JEPA 2: action-conditioned latent prediction



Paper: Assran et al., [arXiv:2506.09985](https://arxiv.org/abs/2506.09985) (V-JEPA 2).

Generative (reconstruction) path

- ▶ Learn latents that **decode** back to pixels (VAE/RSSM).
- ▶ Strength: full **observation model** for planning, denoising, and imagined rollouts with explicit images.
- ▶ Cost: optimization can emphasize pixels over semantics.

Joint-embedding predictive (JEPA) path

- ▶ Predict **embeddings** of future or masked regions in **representation space**.
- ▶ Strength: can learn useful **video/robotics** features without needing to generate every pixel.
- ▶ Cost: no pixel rollouts to inspect, and control requires adding an action pathway (V-JEPA 2-AC, LeWM).

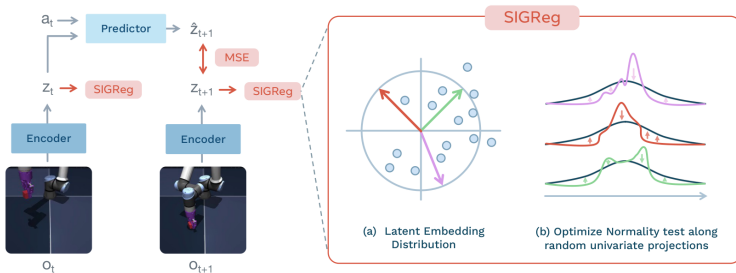


Image credit: Maes et al., [LeWorldModel \(arXiv:2603.19312\)](#).

- ▶ **LeWorldModel (LeWM)**: a JEPA-style world model trained **end-to-end from raw pixels**, presented as the first to do so with only two loss terms.
- ▶ The two terms: a next-latent **prediction loss** and **SIGReg**, a regularizer that pushes latents toward a Gaussian distribution.
- ▶ This replaces the EMA / stop-gradient machinery of earlier JEPA recipes, with a stability and anti-collapse argument (as claimed in the paper).

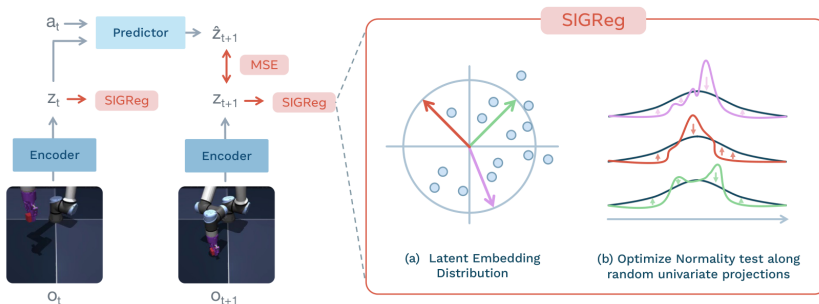
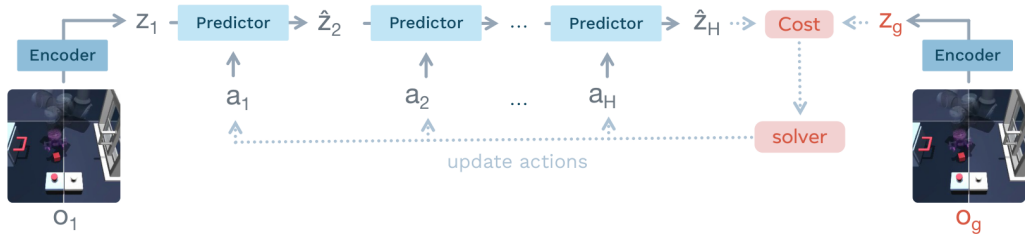


Image credit: Maes et al., [LeWorldModel \(arXiv:2603.19312\)](#).

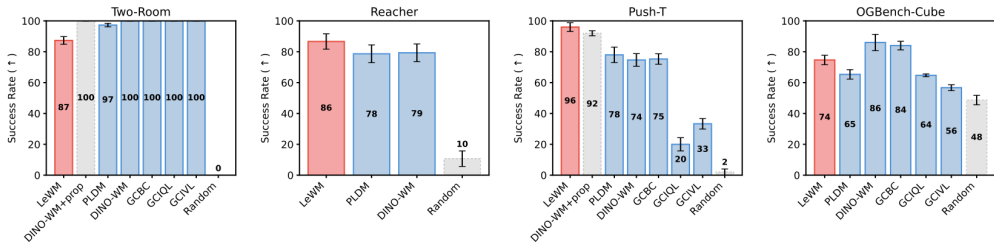
- **Model Architecture.** LeWM consists of an encoder (mapping frames to latent codes) and a predictor (forecasting the next latent given the current latent and action).
- **Training Objective.** The model is trained end-to-end with a prediction loss plus a regularizer that encourages Gaussian-distributed latents:

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda \text{SIGReg}(Z)$$



LeWorldModel Latent Planning. Given an initial observation o_1 and a goal o_g , the world model learned performs planning in the LeWM latent space. The initial state embedding z_1 and the goal embedding z_g are obtained from the encoder. The predictor then rolls out future latent states up to a horizon H . A latent cost between the final predicted state and the goal embedding guides a solver to optimize the action sequence. This prediction–optimization loop is repeated until convergence to a good plan candidate.

LeWorldModel (LeWM): Results



Planning performance across environments (Two-Room, Reacher, PushT, OGBench-Cube). LeWM outperforms PLDM and DINO-WM on PushT and Reacher. DINO-WM leads slightly on the visually complex 3D OGBench-Cube, and both baselines lead on the simple Two-Room task, where the paper suggests SIGReg's Gaussian pressure is a poor fit for a low-dimensional environment.

- ▶ **Ha & Schmidhuber (V-M-C):** modular VAE + MDN-RNN + tiny controller; training inside the dream; a generative model with an evolution-trained policy.
- ▶ **PlaNet / Dreamer v1-v3:** ELBO-trained RSSM latents; CEM planning (PlaNet), then actor-critic learned in imagination (Dreamer); categorical latents and normalization for stability at scale.
- ▶ **V-JEPA 2 (+AC):** non-generative video SSL at scale; an action-conditioned head plus CEM planning brings it to robotics (JEPA lecture).
- ▶ **LeWM:** JEPA from pixels for control, with SIGReg replacing EMA recipes; a small model with fast latent planning, and a good code-first reference for course projects.

- ▶ **Compounding errors:** imagined rollouts drift from reality as the horizon grows; short horizons with value bootstrapping (Dreamer) or frequent replanning (MPC) are workarounds, not solutions.
- ▶ **Model exploitation:** policies find states where the model is wrong. Ha's temperature trick, Dreamer's KL constraints, and SIGReg all target versions of this problem.
- ▶ **Pixels vs. latents, unresolved:** generative simulators produce impressive video with weak physics guarantees; latent models plan well but have no decoder to inspect, so evaluating what they know is indirect.
- ▶ **Goal specification:** reward heads or goal images are the norm; language-conditioned goals for latent world models remain largely open.
- ▶ **Benchmarks are narrow:** games, simulated control, and tabletop robotics dominate; long-horizon, open-world transfer is not demonstrated.
- ▶ **Compute asymmetry:** video-generator simulators are expensive at inference time; latent planners are cheap but lower fidelity. Where the field settles between them is open.

- ▶ World models compress experience so an agent can roll imagined futures and choose actions; the main design split is predict pixels (generative simulators) vs. predict latents (JEPA-style), with hybrid systems in between.
- ▶ Dreamer → DreamerV2 → DreamerV3: a single RSSM-style world model (representation, dynamics, often decode/reward) plus actor-critic training on imagined trajectories; V2 switches to discrete categorical latents; V3 keeps the recipe and stresses stability across many domains with one configuration.
- ▶ JEPA: predict in embedding space, not full observations; V-JEPA scales masked video prediction; V-JEPA 2 adds action-conditioned latent prediction as a bridge to planning and robotics.
- ▶ LeWorldModel: End-to-end from pixels with just next-embedding loss + SIGReg. Fast latent MPC planning with a small model.

- ▶ **Interactive article:** worldmodels.github.io (Ha & Schmidhuber) lets you play with the VAE latents and the dream environment in the browser.
- ▶ **DreamerV3:** the first author's reference reimplementation at [danijar/dreamerv3](https://github.com/danijar/dreamerv3); small control tasks train on a single GPU.
- ▶ **LeWM:** code at [lucas-maes/le-wm](https://github.com/lucas-maes/le-wm); small enough to read end to end, and a good base for course projects.
- ▶ **V-JEPA 2:** checkpoints on Hugging Face (see the JEPA lecture's hands-on slide) for the latent-planning side.
- ▶ **Suggested exercise:** train DreamerV3 on a DeepMind Control task and log imagined vs. real rollouts side by side; then run LeWM's latent planner on PushT and compare how each system fails as the horizon grows.

1. Pedagogical synthesis (informed this revision): *Architectural Evolution of Latent World Models: A Synthesis of Generative and Joint-Embedding Predictive Frameworks* (2026).
2. Schmidhuber, formative work on **curiosity** and agents that build **internal predictive models** (1990s–2000s).
3. Ha & Schmidhuber, *Recurrent World Models Facilitate Policy Evolution* (NeurIPS 2018), [arXiv:1803.10122](#); interactive write-up [worldmodels.github.io](#).
4. Gormley, M., CMU 10-423 / 10-623, **Lecture 26: World Models**. [PDF](#); F25 mirror [PDF \(10423-F25\)](#).
5. Stanford CS25 deck excerpt (**JEPA / Joint Embedding Predictive World Models** slides raster): [JEPA_cs25.pdf \(Google Drive\)](#); course [CS25](#).
6. Cambridge teaching archive (Ha 2018 PDF). [cl.cam.ac.uk PDF](#).
7. Hafner et al., “Learning Latent Dynamics for Planning from Pixels” (PlaNet), ICML 2019, [arXiv:1811.04551](#).
8. Kingma & Welling, “Auto-Encoding Variational Bayes” (VAE), ICLR 2014, [arXiv:1312.6114](#).
9. Hafner et al., *Dream to Control: Learning Behaviors by Latent Imagination* (ICLR 2020), [arXiv:1912.01603 \(Dreamer\)](#). Slides: Figs. 1,3,4,8 cropped from [PDF](#).
10. Hafner et al., “Mastering Atari with Discrete World Models” (**DreamerV2**), ICLR 2021, [arXiv:2010.02193](#). Pedagogical diagrams: Belotti, [Dreamer V2](#) (EclecticSheep blog, 2023).
11. Hafner et al., “Mastering Diverse Domains through World Models” (DreamerV3), [arXiv:2301.04104](#); published as “Mastering Diverse Control Tasks through World Models,” [Nature 2025](#).

12. Bardes et al., “Revisiting Feature Prediction for Learning Visual Representations from Video” (V-JEPA), [arXiv:2404.08471](#).
13. Assran et al., “V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning”, [arXiv:2506.09985](#).
14. Assran et al., “V-JEPA 2.1: Unlocking Dense Features in Video Self-Supervised Learning”, [arXiv:2603.14482](#).
15. Maes et al., “LeWorldModel: Stable End-to-End Joint-Embedding Predictive Architecture from Pixels”, [arXiv:2603.19312](#). PDF [leworldmodel_arxiv2603_19312v1.pdf](#). Code: [github.com/lucas-maes/le-wm](#).
16. Bruce et al., “Genie: Generative Interactive Environments”, ICML 2024, [arXiv:2402.15391](#); Google DeepMind, “Genie 3: A New Frontier for World Models”, 2025 (blog).
17. OpenAI, “Video Generation Models as World Simulators” (Sora technical report), 2024.
18. Hu et al. (Wayve), “GAIA-1: A Generative World Model for Autonomous Driving”, 2023, [arXiv:2309.17080](#).
19. Zhou et al., “DINO-WM: World Models on Pre-trained Visual Features Enable Zero-shot Planning”, 2024, [arXiv:2411.04983](#).
20. Sobal et al., “Learning from Reward-Free Offline Data: A Case for Planning with Latent Dynamics Models” (PLDM), 2025.
21. World Labs, generated interactive 3D worlds. <https://www.worldlabs.ai/>.